TX - SiMDE : Gestion de l'infrastructure informatique

Réalisé par :
Yifan ZHANG
Youssef MOUNHI

Contexte & Objectifs

Contexte

- Le SiMDE héberge des services pour les associations de l'UTC.
- Jusqu'ici, l'hébergement reposait sur l'infrastructure de la DSI. Avec de nouveaux serveurs, l'objectif est de migrer vers une plateforme plus maîtrisée.
- Kubernetes est choisi pour standardiser le déploiement et améliorer la maintenabilité.

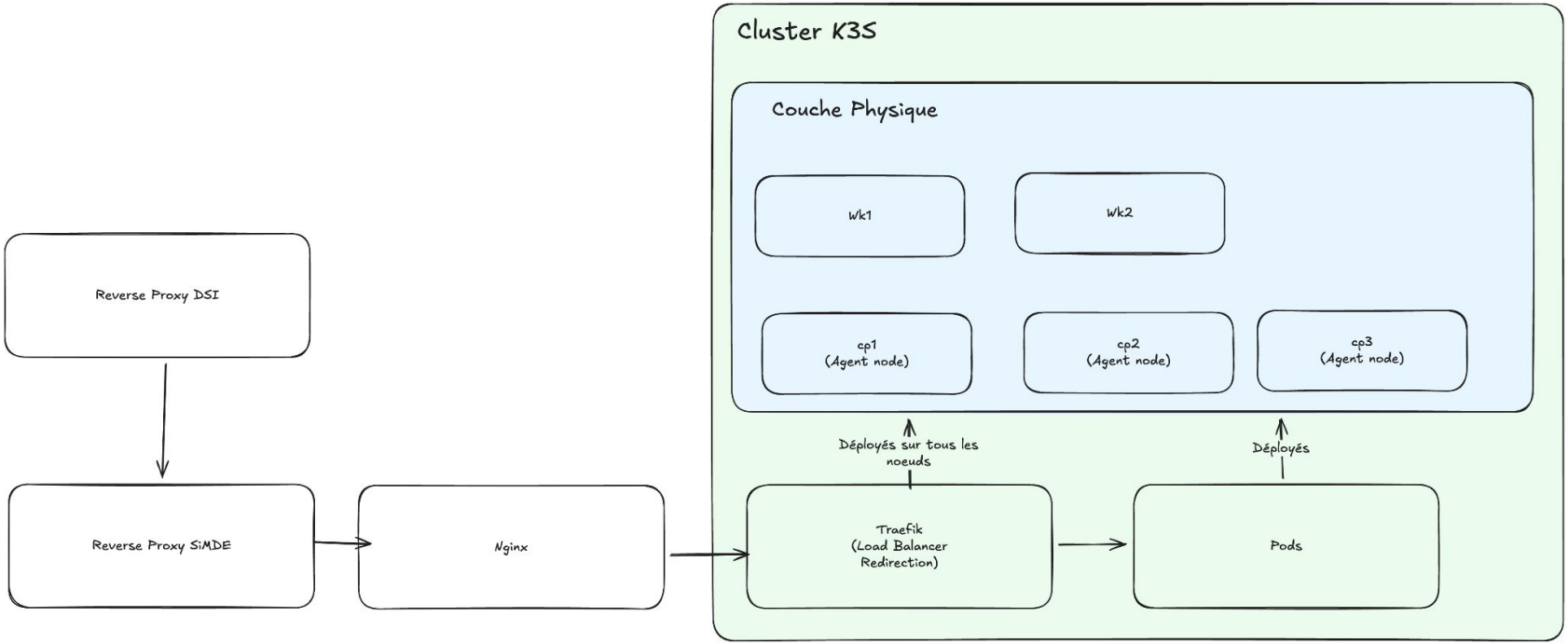
Objectifs

- Apprendre les concepts de Kubernetes et valider le cluster du SiMDE.
- Déployer une application web sur Kubernetes (réseau + configuration + stockage persistant si nécessaire).
- Industrialiser le déploiement : passer de manifests YAML à un chart Helm paramétrable (values / secrets).
- Valider la disponibilité via 3 scénarios : panne d'un worker, panne d'un control-plane, montée en charge (scaling).

Architecture du cluster

- **Cluster Kubernetes (k3s)** sur serveurs physiques du SiMDE
- **Nœuds** : 3 *control-plane* (`cp1`, `cp2`, `cp3`) + 2 *workers* (`wk1`, `wk2`)
- **Réseau / exposition** : accès HTTP via **Traefik Ingress** (IngressClass `traefik`).
- **Stockage persistant** : **Longhorn** (PVC) pour les données applicatives (ex. documents, logs)
- **Bases de données :** déployée en externe
- **Objectif d'architecture** : éviter les points uniques de défaillance (réplication des control-planes et des pods) et faciliter les déploiements standardisés (YAML → Helm)

Architecture du cluster



Comprendre le fonctionnement de Kubernetes

Déploiement d'une application ***Hello-world*** :

- Apprendre les différents concepts de Kubernetes (Déploiement, Ingress, Service, Pods, …)
- Valider que les applications déployées dans le cluster soient accessible depuis Internet
- Se familiariser avec les différents composants de Kubernetes et de Traefik.
- Configurations simples :
  - Déploiement
  - IngressRoute
  - Service
  - Middleware

Déploiement v1 — Manifests Kubernetes (YAML)

Objectif : Déployer Shwet (Laravel) de manière *déclarative* et maîtriser le fonctionnement des ressources Kubernetes.

Ressources principales :

- `Namespace` dédié : **tx-shwet**
- `ConfigMap` + `Secret` : **séparation config / secrets**
- `Deployment` (pod web) + `Service` (ClusterIP)
- `Ingress` (Traefik) : accès HTTP via **shwet-tx.assos.utc.fr**
- `PVC` Longhorn : **documents (2Gi)** + **logs (1Gi)**
- `CronJob` : scheduler Laravel **toutes les 5 min** (`php artisan schedule:run`)

Procédure (ordre) :

1. Namespace + ConfigMap → 2) Secret → 3) PVC → 4) Config PHP-FPM
2. Deployment + Service + Ingress → 6) CronJob

Vérifications :
`kubectl get nodes/pods/endpoints -o wide` + test HTTP (curl/navigateur)

Résultat : application accessible, volumes montés, scheduler fonctionnel — base stable avant migration Helm.

Étude de cas — Shwet (Laravel)

But : déployer une application web *réelle* avec réseau, persistance et tâches planifiées.

Réseau / accès :

- Image : `registry.gitlab.utc.fr/simde/shwet:latest`
- Port applicatif : **8005**
- Service : **ClusterIP shwet:80 → 8005**
- Ingress Traefik : **shwet-tx.assos.utc.fr** (path `/`)

Persistance (Longhorn) :

- `PVC shwet-documents` **2Gi** → `/storage/app/public/documents`
- `PVC shwet-logs` **1Gi** → `/storage/logs`
- **Objectif :** conserver documents et logs après redémarrage / rescheduling.

Étude de cas — Shwet (Laravel)

Configuration :

- `ConfigMap shwet-env` : variables non sensibles (APP_, *DB*_ sans mdp, OAuth URLs…)
- `Secret shwet-secret` : APP_KEY, DB_PASSWORD, clés API/OAuth, mail…

Fiabilité / exploitation :

- Readiness probe : `GET /` sur 8005 (stabilise l'exposition)
- CronJob `shwet-scheduler` : `*/5 * * * *` → `php artisan schedule:run`
  - `concurrencyPolicy: Forbid` + nettoyage TTL

Résultat : application accessible via Ingress, données persistantes via PVC, scheduler opérationnel.

Stockage persistant (PVC) — Longhorn

Pourquoi un PVC ?

- Shwet (Laravel) génère des **fichiers** : *documents uploadés* + *logs*
- Un pod peut être **redémarré / replanifié** → sans stockage persistant, données perdues

Pourquoi Longhorn ?

- Stockage **distribué** adapté aux clusters bare-metal / k3s
- **Provisioning dynamique** via StorageClass (`longhorn`)
- **Réplication** + tolérance à la panne (volume re-attach sur un autre nœud)

Stockage persistant (PVC) — Longhorn

Implémentation (Shwet)

- 2 PVC (StorageClass `longhorn`, `ReadWriteMany`)
  - `shwet-documents` 2Gi → `/var/www/html/storage/app/public/documents`
  - `shwet-logs` 1Gi → `/var/www/html/storage/logs`
- Montés dans le **Deployment** (volumeMounts) avant démarrage de l'app

Débogage (checklist)

- `kubectl get sc` / `kubectl -n tx-shwet get pvc` (Pending vs Bound)
- `kubectl -n tx-shwet describe pvc <name>` (events/provisioner)
- `kubectl -n longhorn-system get pods` + vérifier **volume attaché**
- `kubectl -n tx-shwet describe pod <pod>` (Mount/Attach errors)

Limites du YAML & besoin d’industrialisation

Constat : les manifests YAML “à la main” fonctionnent, mais deviennent compliquées à gérer dès qu’on doit maintenir plusieurs applications.

Principales limites :

- **Duplication** : mêmes patterns (Deployment/Service/Ingress/PVC/CronJob) copiés d’une app à l’autre.
- **Maintenance difficile** : beaucoup de fichiers, risque d’incohérences lors des modifications.
- **Gestion des environnements** compliquée : paramètres différents (image, replicas, host, ressources…) → peu de factorisation.
- **Secrets sensibles** : séparation nécessaire, mais gestion/rotation/versioning restent manuels.
- **Déploiements moins sûrs** : pas d’historique de release, rollback non standardisé.

Conclusion : besoin d’une approche **standardisée et reproductible** → **Helm** (templates + `values.yaml` / `values-secret.yaml`, upgrades & rollbacks).

Déploiement v2 — Helm chart (standardisation)

Objectif : remplacer les YAML statiques par un déploiement **réutilisable, reproductible et maintenable**.

Principe Helm :

- **Templates** Kubernetes + paramètres dans `values.yaml`
- Séparation claire :
  - `values.yaml` : configuration non sensible (image, replicas, ressources, ingress host…)
  - `values-secret.yaml` : **secrets générées depuis un fichier .env** (exclu du dépôt)

Ressources générées par le chart :

- Deployment + Service + Ingress (Traefik)
- ConfigMap (env) + Secret
- PVC Longhorn
- CronJob (Laravel scheduler)
- Labels Helm (suivi des releases)

Déploiement v2 — Helm chart (standardisation)

Structure :
`charts/shwet/` → `Chart.yaml` + `values*.yaml` + `templates/`

Commande unique :

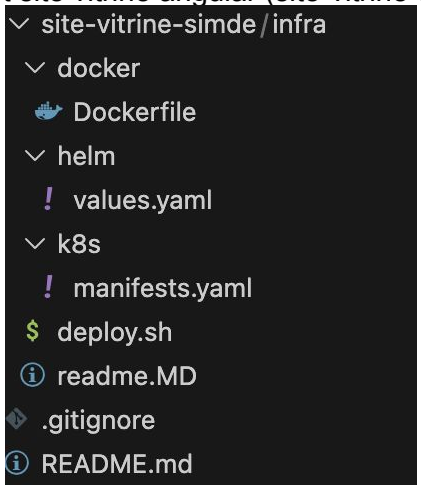
```
helm upgrade --install shwet infra/helm/charts/shwet \
  -n tx-shwet \
  -f infra/helm/charts/shwet/values.yaml \
  -f infra/helm/charts/shwet/values-secret.yaml
```

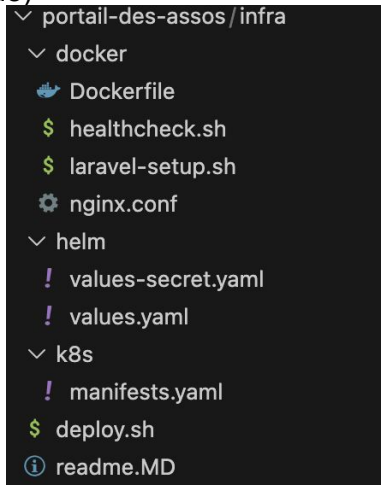
Bénéfices :

- Déploiement standardisé + **mise à jour incrémentale**
- **Historique & rollback** rapides
- Base réutilisable pour d’autres applications SiMDE

Généralisation

- Helm chart — Generic-app
  - Extension de la chart Shwet pour prendre en charges plusieurs types d'applications (testé sur portail-des-assos et site-vitrine-simde)
  - Fournir plus de paramètres de configurations
  - Subchart pour héberger Redis si besoin.
- Fichiers de configuration d'infrastructure de production entiers :
  - Pour : Laravel app (portail-des-assos) et site vitrine angular (site-vitrine-simde)
  - Inclut :
    - Valeurs Helm
    - Configuration docker
    - Script de déploiement





Exemple de fichier de déploiement

```bash
#!/bin/bash
set -e

APP_NAME="portail-des-assos"
NAMESPACE="portail-des-assos"
CHART_PATH="/home/mounhiyo/tx-kubernetes-a25/charts/generic-app"
IMAGE="youssef129/portail-des-assos:latest"

echo "Building Docker image $IMAGE..."
docker build -t $IMAGE -f infra/docker/Dockerfile .

echo "Push the Docker image Docker to the registry..."
docker push $IMAGE

echo "📦 Deploying $APP_NAME..."

# Inject .env to infra/helm/values-secret.yaml
bash $CHART_PATH/scripts/inject-env-in-values.sh

# Deploy
echo "Creating manifests"
helm template $APP_NAME $CHART_PATH \
  -f ./infra/helm/values.yaml -f ./infra/helm/values-secret.yaml \
  -n $NAMESPACE > ./infra/k8s/manifests.yaml
echo "Applying manifests..."
kubectl apply -f infra/k8s/manifests.yaml -n $NAMESPACE

echo "✅ Deployed successfully!"
echo ""
kubectl get pods -n $NAMESPACE
```

Tests de résilience — 3 scénarios (disponibilité)

Méthode commune (preuves) :

- Avant / après : `kubectl get nodes/pods/endpoints -o wide`
- Vérification service : test HTTP (curl / navigateur)
- Objectif : observer **rescheduling**, **temps de reprise**, et **impact utilisateur**

Scénario 1 — Panne d'un worker (wk1/wk2)

- Action : arrêt brutal d'un nœud worker
- Attendu : pods replanifiés sur un nœud sain (si ≥ 2 replicas)
- Observation : indisponibilité courte, puis service rétabli

Tests de résilience — 3 scénarios (disponibilité)

Scénario 2 — Panne d'un control-plane (cp1/cp2/cp3)

- Action : arrêt d'un control-plane
- Vérification : API toujours disponible (`/readyz?verbose`)
- Résultat : cluster opérationnel, applications déjà lancées restent disponibles

Scénario 3 — Variation de charge (scaling)

- Action : augmentation du nombre de replicas (`kubectl scale` / Helm values)
- Attendu : nouveaux pods créés et répartis, endpoints mis à jour
- Résultat : service maintenu pendant la montée en charge

Tests de résilience — 3 scénarios (disponibilité)

Conclusion : le cluster tolère la perte d'un nœud et supporte le scaling ; la redondance + la réplication améliorent la disponibilité.

- Mécanisme de hearbeat pour détecter l'état des nœuds. L'intervalle entre chaque heartbeat est par défaut défini à 40s.
- Timeout pour la recréation des pod qui est par défaut égal 5 min (paramètre : ***pod-eviction-timeout*** ou ***tolerationSeconds)*** : Pod status -> Terminated -> réplication sur un autre noeud disponible.
- Le temps d'indisponibilité est grandement dû à la configuration Nginx en point d'entrée : temps pour rediriger le trafic vers un autre noeud.
- Scaling via la réplication de pods en fonction de la variation de charge.

Livrables

Infrastructure as Code

- Déploiement **YAML** complet (Namespace, Deployment, Service, Ingress Traefik, PVC Longhorn, CronJob)
- **Chart Helm** pour Shwet + app général: templates + `values.yaml` / `values-secret.yaml`

Scripts & configurations

- Scripts de déploiement / healthcheck / initialisation applicative
- Configurations d'entrée (ex. Dockerfile, Nginx, laravel setup) et paramètres associés

Documentation

- README de déploiement (YAML → Helm), procédures d'installation / mise à jour / vérification
- Rapport détaillant les réalisations de la TX.

Preuves de tests

- Résultats et captures des 3 scénarios de résilience (worker down, control-plane down, scaling)

Améliorations possibles

- **GitOps :** Installer ArgoCD
  - **Objectif :** Automatiser le déploiement et garantir la synchronisation entre Git et le cluster
  - **Déploiement déclaratif** : Git comme source unique de vérité
  - **UI :** interface graphique pour suivre l'état des déploiements
  - **Rollback simplifié**
  - **Historique des changements**
  - **Détection des changements manuel appliqués directement sur le cluster.**
  - **Comment ?**
    - Installé dans le cluster Kubernetes en tant que Controller.
    - Gestion des clés secrètes avec des outils dédiés (sealed secrets, vaults)
    - Détecte les changements des ***values.yaml*** lors des mises à jour.
  - **Environnements multiples (dev/staging/prod)**
- **Remplacer Nginx en point d'entrée par un Load balancer (ex: MetalLB)**

Conclusion

Ce que nous avons réalisé

- Déploiement complet en **YAML Kubernetes** (Namespace, Deployment, Service, Ingress Traefik, ConfigMap/Secret, PVC Longhorn, CronJob)
- Intégration de **Shwet / PDA / etc (Laravel)** : accès HTTP + persistance (documents/logs) + scheduler fonctionnel
- Migration vers **Helm** : templates + `values.yaml` / `values-secret.yaml` → déploiement reproductible (upgrade / rollback)

Résilience validée (3 scénarios)

- Panne **worker** → rescheduling + service rétabli
- Panne **control-plane** → cluster opérationnel (`/readyz`)
- **Scaling** → nouveaux pods + service maintenu